

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

Develop an application that will evaluate a stock in terms of the amount of profit it can potentially offer if you buy and later sell it. If the stock is valued at b dollars on day x and s dollars on a subsequent day y , if you bought stock on day x and sold it on day y you would make a profit of $s - b$ dollars.

You want to write an algorithm that will analyze retrospective price data data to compute the *maximum* profit you could acquire by buying and selling the stock at the right time. The data is in the form of an array P with n entries, where $P[i]$ denotes the price of the stock on day i . Given P , your algorithm should return the maximum possible profit, i.e., the maximum $P[i] - P[j]$ where $i > j$.

Design an efficient algorithm to solve this problem.

Example

Input: $P = [10, 17, 4, 12, 13, 13, 11, 14, 3, 12, 8, 2, 9]$

Output: 10

(since $P[8] - P[3] = 14 - 4 = 10$, and no other pair gives a difference more than 10)

Solution-1: Divide and Conquer

General Idea

The key observation is that the maximum profit for an array split into two subarrays (left and right) can be achieved in one of three ways:

- Buying and selling within the left subarray.
- Buying and selling within the right subarray.
- Buying in the left subarray and selling in the right subarray.

To compute the maximum profit, we recursively divide the array into two halves and compute:

- The maximum profit in the left half.
- The maximum profit in the right half.
- The maximum profit achieved by buying at the minimum price in the left half and selling at the maximum price in the right half.

By keeping track of the minimum price, maximum price, and maximum profit in each subarray during recursion, we can compute the maximum profit for the entire array.

Pseudocode

Algorithm 1 Divide and conquer algorithm to find the maximum profit

```

1: procedure MAXPROFIT( $P, l, r$ )
2:   if  $l == r$  then
3:     return  $(0, P[l], P[l])$  ▷ max profit, min price, max price
4:   end if
5:    $mid \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
6:    $(left\_prof, left\_min, left\_max) \leftarrow \text{MAXGAIN}(P, l, mid)$ 
7:    $(right\_prof, right\_min, right\_max) \leftarrow \text{MAXGAIN}(P, mid + 1, r)$ 
8:    $cross\_prof \leftarrow right\_max - left\_min$ 
9:    $max\_profit \leftarrow \max(left\_prof, right\_prof, cross\_prof)$ 
10:   $min\_price \leftarrow \min(left\_min, right\_min)$ 
11:   $max\_price \leftarrow \max(left\_max, right\_max)$ 
12:  return  $(max\_profit, min\_price, max\_price)$ 
13: end procedure
14: procedure FINDMAXGAIN( $P$ )
15:   $(max\_profit, -, -) \leftarrow \text{MAXPROFIT}(P, 1, n)$ 
16:  return  $max\_profit$ 
17: end procedure

```

Proof of Correctness

Base Case: $n = 1$

If there is only one element in the array (i.e. $l = r$), then the algorithm correctly returns $(0, P[l], P[l])$. This is correct because with only one price point, no valid transaction (buying and selling at different times) can be made, so the maximum profit is 0. There is only one value, so that value is trivially the maximum and minimum value in that subarray.

Inductive Hypothesis: Assume that for any array of size $k \leq n - 1$, the MAXPROFIT algorithm correctly computes the maximum possible profit, the minimum price, and the maximum price within that range.

Inductive Step: Consider an array of size $n = k + 1$ with indices ranging from l to r . The algorithm recursively divides the array at the midpoint $mid = \lfloor (l + r)/2 \rfloor$ into two subarrays:

- The left subarray $P[l \dots mid]$
- The right subarray $P[mid + 1 \dots r]$

We have 3 possible options for how we achieve the maximum profit for our current array $P[l \dots r]$:

1. The maximum profit is contained within the left subarray.

By the inductive hypothesis, the algorithm correctly computes the maximum gain for the left subarray $P[l \dots mid]$

2. The maximum gain is contained within the right subarray.

By the inductive hypothesis, the algorithm correctly computes the maximum profit for the right subarray $P[mid + 1 \dots r]$

3. The maximum gain spans the two subarrays — the minimum is on the left and the maximum is on the right.

To achieve the maximum profit spanning two subarrays, we use the smallest element in the left subarray and the largest element in the right subarray. This is done by subtracting the minimum in the left subarray from the maximum in the right subarray. By the induction hypothesis, those values are computed correctly since the subarrays are both smaller than k .

The overall maximum profit is computed as the maximum of the three options above. The minimum and maximum values in $P[l \dots r]$ are computed by calculating the minimum and maximum of the minimums and maximums of the two halves.

Thus, by induction, the MAXPROFIT algorithm correctly computes the maximum profit, minimum value, and maximum value for any input array of size n .

Runtime Analysis

Master Method

Let $T(n)$ denote the runtime of MAXPROFIT on an array of size n . The function recursively divides the array into two halves, computes the result for the left half ($T(\frac{n}{2})$) and the right half ($T(\frac{n}{2})$), then combines these results in constant time $\Theta(1)$.

Putting it all together, the work done by the algorithm can be represented by the recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1)$$

Using the Master Method, we have $a = 2$, $b = 2$, and $f(n) = \Theta(1)$.

Therefore, we have $g(n) = n^{\log_b a} = n^{\log_2 2} = n$. Since $f(n)$ is polynomially smaller than $g(n)$, we find that Case 1 of the Master Theorem applies, giving:

$$T(n) = \Theta(n)$$

Solution-2: Iterative

General Idea

The key idea of the iterative solution is to traverse the array while keeping track of the minimum price encountered so far, and calculating the potential profit at each step.

For each day i , we calculate the difference between the current price $P[i]$ and the minimum price seen so far. This gives the potential profit for selling on day i and buying at the minimum price before that day. We update the maximum profit whenever we find a larger profit. We only need to make a single pass through the array.

Pseudocode

Algorithm 2 Iterative algorithm to find the maximum profit

```
1: procedure FINDMAXPROFITITERATIVE( $P$ )
2:    $min\_price \leftarrow \infty$ 
3:    $max\_profit \leftarrow 0$ 
4:   for  $i \leftarrow 1$  to  $n$  do
5:      $min\_price \leftarrow \min(min\_price, P[i])$ 
6:      $max\_profit \leftarrow \max(max\_profit, P[i] - min\_price)$ 
7:   end for
8:   return  $max\_profit$ 
9: end procedure
```

Proof of Correctness

Loop Invariant: At the start of each iteration i of the loop:

- min_price is the minimum value in the subarray $P[1 \dots i - 1]$.
- max_profit is the maximum possible profit that can be achieved by buying and selling before day i . In other words, it is the maximum profit in the subarray $P[1 \dots i - 1]$.

Initialization: Before the loop starts ($i = 1$), min_price is ∞ and max_profit is 0. The subarray $P[1 \dots 0]$ is empty, and the minimum value and maximum profit are the maximum and minimum possible values respectively, so the loop invariant holds.

Maintenance: Assume the loop invariant holds at the start of iteration $i - 1$. Consider the i th iteration.

- If $P[i] < min_price$, then min_price is updated to $P[i]$. By our assumption, min_price holds the minimum value seen in $P[1 \dots i - 1]$, so by comparing it to $P[i]$ and taking the minimum, it now holds the minimum value in $P[1 \dots i]$.
- Because min_price correctly stores the minimum value encountered in $P[1 \dots i]$, we can calculate the maximum gain if we sell on day i as $P[i] - min_price$. We update max_profit as the maximum of the maximum profit achievable in $P[1 \dots i - 1]$ (correctly computed due to the loop invariant) and the maximum gain if we sell on day i . Therefore max_profit is calculated so that it is the maximum profit in the subarray $P[1 \dots i]$.

Termination: The loop terminates when $i = n + 1$. At this point, max_profit stores the maximum profit achievable in $P[1 \dots n]$. Therefore, the algorithm correctly computes the maximum profit in P .

Runtime Analysis

The algorithm processes each element of the array exactly once. For each element, the algorithm performs constant-time operations: updating the minimum price and calculating the potential gain.

Therefore the time complexity is:

$$T(n) = \Theta(n)$$

Presentation 2

Given a 2-dimensional array A of size $n \times n$, we would like to find the index (i, j) of a *locally minimum* value. All numbers in the array are distinct. An entry $A[i, j]$ is locally minimal if it is smaller than all of its four neighbors ($A[i, j] < A[i+1, j]$, $A[i, j] < A[i-1, j]$, $A[i, j] < A[i, j-1]$ and $A[i, j] < A[i, j+1]$). Neighbors that are out of bounds are not considered, e.g. $A[1, n]$ is locally minimal if $A[1, n] < A[2, n]$ and $A[1, n] < A[1, n-1]$ as the indices $(0, n)$ and $(1, n+1)$ are out of bounds.

Example Input:

$$\begin{pmatrix} 5 & 15 & 9 & 0 & 6 \\ 3 & 1 & 2 & 13 & 14 \\ 4 & 7 & 23 & 19 & 25 \\ 29 & 17 & 24 & 35 & 27 \\ 37 & 16 & 45 & 47 & 8 \end{pmatrix}$$

Output: $(0,0) = 5$, $(0,3) = 0$, $(1,1) = 1$, $(1,4) = 4$, $(4,1) = 16$, $(4,4) = 8$
(Returning **any** of these pairs would be a valid solution to the problem.)

Solution:

We can use a divide-and-conquer approach to solve this problem. We first consider the middle two rows and middle two columns of the grid, and find the smallest value among those elements. The quadrant that contains the smallest element is guaranteed to have a locally minimal element. We can then recurse on that quadrant until we find the locally minimal element.

Given a 2D array A of size 4×4 , the middle two rows and middle two columns are in yellow, the quadrant identified is in green, and the minimum found is in blue:

$$A = \begin{bmatrix} 8 & 7 & 6 & 5 \\ 9 & 3 & 4 & 1 \\ 10 & 11 & 2 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Pseudocode

```
1: procedure FINDLOCALMINIMUM(grid)
2:    $n \leftarrow$  size of grid
3:   if  $n \leq 3$  then
4:     Exhaustive search for local minimum
5:   else
6:      $middleRows, middleCols \leftarrow$  middle two rows and columns of grid
7:      $smallestVal \leftarrow$  minimum value in middleRows and middleCols
8:      $q \leftarrow$  quadrant of grid containing smallestVal
9:     return FindLocalMinimum(q)
10:  end if
11: end procedure
```

Proof of Correctness:

Base Case:

For small grids of size $n \leq 3$, we perform an exhaustive search to find a local minimum. Since there are at most 9 elements in a 3×3 grid, we can check each element against its neighbors in constant time. Thus, the algorithm is trivially correct for these cases.

Inductive Hypothesis: Assume that for any grid of size $k \times k$ where $k < n$, the algorithm correctly finds a local minimum.

Inductive Step: For an $n \times n$ grid:

The algorithm searches the middle two rows and middle two columns, identifying the smallest value among these elements. Call this value $A[i, j]$. The algorithm recurses into the quadrant that contains $A[i, j]$. Consider the (absolute) minimum element in that quadrant. If it is an interior element, then it is trivially also a locally minimal element. If it is a border element, then it *might* not be locally minimal (because there may be a neighboring larger element outside the quadrant). However, we chose the quadrant that contains the smallest border element $A[i, j]$. Since we considered the middle two rows and columns, all of the neighbors of $A[i, j]$ that are not in the quadrant were examined and found to be larger than $A[i, j]$. Therefore, no border element outside the quadrant will be $> A[i, j]$. The quadrant the algorithm searches is therefore guaranteed to have a locally minimal element. By the inductive hypothesis, the algorithm correctly finds a local minimum in the quadrant. Therefore, by induction, the algorithm correctly finds a local minimum for any $n \times n$ array.

Runtime Analysis

Master Method

We can find the smallest element in the middle two rows and middle two columns of the grid in $\Theta(n)$ time.

Since a maximum of one recursive call is made per function (picking one of the remaining four quadrants) and n is reduced by $\frac{1}{2}$ at each call, the runtime can be modeled with the following recurrence equation:

$$T(n) = T(n/2) + \Theta(n)$$

In this case, $a = 1$, $b = 2$, $f(n) = \Theta(n)$, $g(n) = n^{\log_b a} = n^0$. Since $f(n)$ is polynomially larger than $g(n)$, we encounter Case 3 of Master Method. Therefore the runtime is $\Theta(n)$.

Recursion Tree

At each level of recursion, we perform $\Theta(n)$ work outside of the recursive call, and the problem size reduces by half.

- Level 0: $\Theta(n)$
- Level 1: $\Theta(n/2)$
- Level 2: $\Theta(n/4)$
- Level k: $\Theta(n/2^k)$

The recursion stops when $n/2^k = O(1)$, meaning the depth of the recursion tree is:

$$k = \log_2 n$$

The total work done across all levels is:

$$\sum_{i=0}^{\log_2 n} \Theta(n/2^i) = \Theta(n) \sum_{i=0}^{\log_2 n} \frac{1}{2^i}$$

Since the sum of this geometric series converges to a constant, the total work simplifies to $\Theta(n)$.